# Runway

## The complete guide: what we built, how it works, and how to show it

Sheridan Datathon 2026 · Tangerine track · Team Runway (Sukhman Aulakh, engine lead)

Written November 8, 2026, from the code on main and evidence.json. Every number in this guide is a tested result from the data, not an estimate.

### How to read this guide

| Part | For | What you get |
|---|---|---|
| 1. The simple version | Anyone, including you at 2 a.m. | What Runway does, explained with one customer as the example |
| 2. The words | Anyone | Every term on screen in plain English |
| 3. The product, screen by screen | You, before the demo | What each screen shows, what to click, what to say |
| 4. The business logic | Bankers, consultants | Who gets which action, and why it makes money or saves it |
| 5. The technical guide | Engineers, the professor | Architecture, data, every calculation, statistics, AI, API, tests, Docker and cloud |
| 6. What you built | You, for Q&A and interviews | The work, round by round |
| 7. Showing it to the judges | You, on the day | The 5-step pitch, the live twist, what to touch, the hard questions |
| 8. Is it ready to ship? | You and a CTO | Checklist, what is and isn't production-grade |
| 9. Strict rating and ranking | You | Every component scored, and where we honestly stand |

# 1. The simple version

## The idea in one sentence

> Banks look at your **average** balance. Overdrafts happen at your **lowest** balance. Runway watches the low point, finds the customers who are about to go below $0 before they do, and tells the bank exactly what to do about each one.

## A story with one real customer: account 93172

Set the Time Machine to December 2024 and open customer 93172. Here is what the bank sees, step by step.

1. **Looks fine.** Their average balance over 6 months is **$18,503**. The standard runway formula (the one the judges gave us) says **no shortfall (120+ months)**. On paper, this person is safe forever.
2. **The low point.** But inside the month their balance dipped to **$238**. Lowest ÷ average is about **1%**: riskier than **98%** of customers. A banker would call it a **99% haircut**: almost all of the average is gone at the bottom of the month.
3. **What changed.** That same month the bank lent them **$166,958**, with a payment of **$4,638 a month**. The standard formula can't see it yet, because it only looks at past spending. Our realistic runway counts it: **2.4 months**, not 120+.
4. **What happened next.** They went overdrawn in **January 2025**. Our calibrated forecast had given them a **13.8%** chance of a first overdraft in the next 12 months (about 8 times the 1.6% average).
5. **What Runway tells the advisor to do.** Money in fell 27% ($14,323 a month vs $19,627 before), so: **call first this week**. Then a free alert when the balance falls below **$6,527** (their own typical dip, not a generic $100), and an automatic transfer from their own savings of the same size. A small overdraft limit comes last, and only after a call, because it costs them money (about **$108** for a month at Tangerine's own terms).
6. **And the loan?** A simple check at approval, using only data up to November 2024, would have **held that loan for review**: they were already in the riskiest 10%.

## Why that matters, in three numbers

- If the bank flags the riskiest **10%** by the low point, it catches **34 of 55** first overdrafts the next year (62%). This held in 2023, 2024 and 2025, each tested on a year the model never saw.
- Every one of the **76** borrowers who was ever overdrawn defaulted on their loan. Overdrafts are an early warning for credit losses.
- A review rule at loan approval holds **17%** of loans; those loans default **34.7%** of the time vs **6.2%** for the rest.

## What Runway is not

- It is not an automatic credit decision. A human advisor acts on every output.
- It does not use AI to calculate. Every number comes from the engine; AI only reads twist cards and writes sentences, and every number it writes is checked.
- It is built on 1990s Czech bank data (re-dated to 2020–2025), not real Tangerine customers. That's why a real rollout starts in shadow mode.

# 2. The words you need

| Word on screen | Plain English | Example (93172, Dec 2024) |
|---|---|---|
| Money in / money out | Everything that arrived in the account in a month / everything that left it | About $18,301 in and a similar amount out per month |
| Standard runway | The judges' formula: months until savings run out after a shock, using medians. We never change it (it's the answer key) | No shortfall (120+) |
| Realistic runway | Our version: last 12 months, EI capped at $729/week and ending, pensions kept, the in-month dip, and new loan payments counted | 2.4 months |
| Liquidity trough | Lowest balance ÷ average balance over 6 months. Near 0% = the account keeps dipping close to $0 | 1% |
| Haircut | 1 − trough: the share of the average balance that's gone at the low point (bankers' word) | 99% |
| Riskiest 10% | Customers whose trough is in the bottom 10% of all customers with 12+ months of history | Riskier than 98% |
| Calibrated risk | The chance of a first overdraft in 12 months, from the simulation, corrected so 1.6% really means 1.6% | 13.8% |
| Twist card | The judges' surprise scenario (job loss, rent up, a repair). We turn its words into settings | Job loss + bills +15% |
| Verdict | Yes / No / Only on paper / Fragile: can they make it 6 months, and the biggest reason | — |
| Simulated futures | 2,000 possible next years, built from the customer's own past months | “7 in 10 futures stay above $0” |
| Cushion / buffer | Money that should sit ready to cover the in-month dip (an auto-transfer or a small limit) | $6,527 (conservative $10,647) |
| Alert level | When the balance falls below this, warn the customer. Set at their own dip | $6,527 |
| Tier | Which group a customer is in: already overdrawn, act now, watch, stable, new | Act now |
| Hazard ratio | How many times faster something happens to one group than another | Flagged: 11–12× faster first overdrafts |
| AUC | How well a score ranks risky above safe customers. 0.5 = coin flip, 1.0 = perfect | 0.86 for the trough in 2025 |
| Out of time | Built on one year, tested on the next. The honest way to test a forecast | Fit on 2024, judged on 2025 |

# 3. The product, screen by screen

Runway is one web app with five screens down the left side. Across the top: the **account box** (type any of 4,500 accounts), the **Time Machine** (Today = Dec 2025, Dec 2024, Dec 2023: every number is cut to that date, so nothing looks into the future), **Pitch mode**, and the **AI badge** (Gemini on Vertex AI, or offline rules).

## 01 Who's at risk (the bank's view)

| What you see | What it means | Engine source |
|---|---|---|
| Headline numbers | Riskiest 10% → 34 of 55 overdrafts; 13–18× likelier; alert 3 months early | evidence.json → trough, impact |
| The whole book | Every customer in one tier with one action and the rate that tier really overdrew the next year | book_tiers, evidence → book |
| Advisor call list | The riskiest customers this month, each with their own cause and next step; click a row to open them | watchlist + diagnose |
| If everyone lost their job | Share of all customers still above $0 by month: standard vs realistic | portfolio_curve |
| Check the trough before lending | The loan-approval rule and its backtest | evidence → loan_policy |
| Who the riskiest are | Not poor: higher money in, more loans, almost no pensioners | evidence → segment |

## 02 Know the customer

| What you see | What it means | Engine source |
|---|---|---|
| 4-step story strip | Looks fine → the low point → what changed → act. Builds itself for any account | build_context |
| Chance of a first overdraft | Calibrated 12-month risk (only for customers never overdrawn, 12+ months) | overdraft_risk |
| Liquidity trough card | Trough, percentile, haircut | liquidity_trough |
| Standard vs realistic runway | The judges' number next to ours | runway, realistic_runway |
| Why they're at risk, and what to do | Causes in order of urgency, each with an action sized from their own numbers | diagnose |
| When? | Among customers like them, how many had a first overdraft within 6 / 12 / 24 months | when_overdraft (survival) |
| Advisor assistant | Brief me before I call; draft a message to the customer in English or French | ai.py (words only) |
| Balance chart and what happened next | The real months; in Time Machine mode, what actually happened after | history, what_happened_next |
| Loan card | Their loan, its payment, and whether the approval check would have held it | loan_check |

## 03 Stress test (the live twist)

1. Type or pick a twist card (for example “They lose their job and rent goes up 15%”).
2. Check the **chips**: the settings the AI read from the card. If the AI and the offline rules disagree on a field, it's flagged. Confirm before running.

3. Read the **verdict**: can they make it 6 months? Yes / No / Only on paper / Fragile, and **the biggest reason in months**.
4. Below: both runways, why they differ, the simulated futures (fan chart), and for risk teams an exposure profile (labelled uncalibrated).

## 04 Make a plan

- **Do this first**: the cause (for 93172, money in fell: call this week).
- **Then cover the dip**: 1. alert (free), 2. auto-transfer from savings (free), with a conservative option, 3. move money to chequing now. The cost of a limit at Tangerine's terms is shown, which is why it comes last.
- **Does a dip-sized cushion work?** The 2025 backtest, plus the risk-appetite table (median vs 75th percentile vs largest dip).
- Smallest cuts to stay out of overdraft (a linear program), a backup plan ranked by months gained, emergency-fund milestones, and an AI-written plan in plain English.

## 05 Why trust it

Headline proof; our runway vs the judges' formula (tested); model health (10 monitors, no breaches); the 3-year backtest; calibration; customers steer back; survival (when first overdrafts come); champion vs challenger; loan rule; fairness; who the riskiest are; which reasons hold up; what didn't hold up and what we did; limits with the production fix.

## Pitch mode

Press **Pitch mode** in the top bar. It drives the app through five steps; move with Next/Back or the arrow keys, Esc to leave. Each step shows a short cue of what to point at. The cues contain no numbers, so they can never contradict the screen.

# 4. The business logic

## Who it's for

A bank's **customer-care advisors** (who to call this week, why, and what to offer) and its **risk teams** (how the book behaves, whether a loan should be reviewed). The customer benefits through them: fewer overdrafts, fewer fees, earlier help.

## The whole book: every customer gets one tier

| Tier | Rule (data up to the as-of month) | Action | Overdrew next year (Dec 2024 → 2025) |
|---|---|---|---|
| Already overdrawn | Any day below $0 in the last 12 months | Recovery plan + personal alert | 60% |
| Act now | Trough in the riskiest 10% | Advisor contact, matched to the cause | 12.7% |
| New and swinging | 3–11 months old, top 10% of new accounts by swings in money in | Welcome call + personal alert | 18.6% |
| Watch | Trough in the next 20% | Automatic personal alert, no call | 2.7% |
| Stable | Everyone else | Nothing | 0.4% (same every year) |
| New | Under 3 months, or new but steady | Standard alerts | 2.8% |

> The headline for a banker: the top three tiers are about **10%** of customers and produce **68%** of next year's overdrafts. More than half the book needs **nothing**.

## The action ladder: safest first

1. **Call** when there is a cause to fix (money in fell → income check, EI, payment holiday; a heavy new loan → payment date or longer term).
2. **Alert** at the customer's own danger level. Free, no credit risk. In 2025 it fired before **23 of 34** caught overdrafts, a median **3 months** early (a generic $100 alert: 1 of 34).
3. **Auto-transfer** from their own savings, sized to their typical dip. Free. Covered **27 of 34** caught overdrafts in full; only **$78K** of **$2.9M** was ever drawn: it works as a limit, not cash.
4. **A small overdraft limit**, only after a call, priced at Tangerine's terms ($5 a month used + 19% a year). It comes last because every overdrawn borrower in this data defaulted.

## Risk appetite: how big should the buffer be?

| Buffer set at | Overdrafts covered, 2025 | Limits offered | Actually drawn |
|---|---|---|---|
| Median dip (default) | 27 of 34 | $2.9M | $78K |
| 75th percentile (conservative) | 29 of 34 | $3.8M | $83K |
| 90th percentile | 29 of 34 | $4.8M | $89K |
| Largest dip | 30 of 34 | $6.1M | $95K |

The 75th percentile is the knee: two more customers covered for about $5K more drawn. The bank chooses; we show the trade-off.

## Lending: a check at approval

Hold a loan for review if, using only data before the loan month, the applicant was already overdrawn or is in the riskiest 10% by trough. Nothing is fit on loan outcomes. Result on all 682 loans: holds **118 (17%)**, catches **41 of 76** defaults, held loans default **34.7% vs 6.2%** (counting only loans that finished unpaid: **10.2% vs 3.4%**, p = 0.003). It also holds 77 good loans, so “review” means talk first (smaller loan, cushion, alert), not reject. Dollar figures are amounts lent, not losses.

## Why a bank would pay for this

- **Fewer overdrafts and fees** for customers (the federal NSF fee cap is now $10, so fee income is shrinking anyway; preventing overdrafts protects the relationship).
- **Fewer credit losses**: the same signal flags risky loans at approval.
- **Advisor time spent where it matters**: a call list of about 10% of customers, each with a reason.
- **Regulation-ready**: a model card in OSFI E-23 format, monitoring with thresholds, fairness checks, a human in the loop.

## The pilot plan (what a consultant asks)

| Phase | What happens | Success measure |
|---|---|---|
| 0–3 months: shadow | Score the whole book monthly; no customer contact; compare flags with real overdrafts | Catch rate and monitors stay within thresholds on Tangerine's own data |
| 3–6 months: controlled pilot | Half of flagged customers get call + alert + transfer; half business as usual | First-overdraft rate, complaints and fees, treated vs control (this proves cause) |
| 6+ months: scale | Call list in the CRM; loan review at origination | Overdrafts and loan losses avoided; advisor time per call |

## Finance terms: what we added and what we refused

| Term | Decision | Why |
|---|---|---|
| Haircut | Added | It's the trough in a banker's words: 93172 takes a 99% haircut at the low point |
| Confidence level / risk appetite | Added, backtested | The buffer table above |
| Volatility | Already in | Cash-flow swings pick the auto-save action; swings in money in score new accounts |
| Probability of default (PD) | Have it | Calibrated risk: predicted 1.61% vs 1.61% actual in 2025 |
| Exposure | Have it | Money drawn on the buffer; exposure profile on the Stress screen |
| Expected loss (PD × EAD × LGD) | Refused | Loss given default isn't in the data; we won't invent it |
| Portfolio VaR / correlation | Refused | No shared shocks in the data to estimate correlation; the bank-wide job-loss curve shows one shared shock instead |

# 5. The technical guide

## Architecture

One rule shapes everything: **all maths lives in engine.py**. Everything else only moves or displays numbers.

| Layer | File(s) | Job |
|---|---|---|
| Data | data/*.csv (never committed) | 9 Tangerine/PKDD'99 files: 4,500 accounts, 185,326 account-months, 1,056,320 transactions, 682 loans |
| Engine | engine.py | Every calculation: both runways, trough, simulation, calibration, verdict, cushion, diagnosis, loan check, tiers, survival, optimiser |
| Evidence | evidence.py → evidence.json | Every backtest and statistical test; rebuilt with python evidence.py |
| Context | ui/context.py, ui/data.py | Builds one customer's numbers (shared by both front ends); caches anything that loops over all accounts |
| AI | ai.py | Twist-card reader and plan/brief/message writer, with number and wording checks; Gemini on Vertex AI or offline |
| API | api.py (FastAPI) | JSON endpoints for the dashboard; also serves the built React files |
| Dashboard | web/ (React 19, Vite, TanStack Query, Recharts) | The five screens and Pitch mode; never calculates money |
| Backup app | app.py (Streamlit) | Same engine, older UI; used if the React build fails |
| SQL | sql/trough.sql | The trough as one warehouse query; tested identical to the Python |
| Ship | Dockerfile, docs/DEPLOY.md, scripts/smoke.py | One container for Cloud Run; smoke test against any URL |

## How one request flows

1. The browser asks GET /api/customer/93172?asof=2024-12.
2. api.py loads the data once (cached), cuts it at 2024-12 and calls ui/context.build_context.
3. build_context calls engine functions: profile, runway, realistic_profile, liquidity_trough, overdraft_risk, cushion, diagnose, loan_check, when_overdraft…
4. The API returns JSON; the React screen formats and draws it. If the user asks for words, ai.py gets the engine's facts and must reuse them exactly.

## Every calculation, defined

| Name (engine) | Definition |
|---|---|
| Judges' runway (profile, runway) | I = median money in, S = median money out, B = mean average daily balance over the last 6 rows, H = median household bills, F = median bills + insurance + loans. Runway = (B − one-off) ÷ (spending after shock − income after shock); 0 if buffer ≤ 0; “no shortfall (120+)” if the gap ≤ 0; cap 120; 1 decimal. Answer key: 20111 → 3.4 / 4.0 / 3.4 months. |
| Realistic runway (realistic_profile, | Last 12 months' means; pension and interest kept in a job loss; EI 55% capped at $729/week (≈ $3,159/month) and stopping after its months; usable buffer = B |

| Name (engine) | Definition |
|---|---|
| realistic_runway) | − typical in-month dip; a loan granted in the window counts at its full payment. |
| Liquidity trough (ltr_table) | Lowest ÷ average balance over 6 months; scored with 12+ months of history; percentile rank, 0 = most fragile; flag = bottom 10%. Haircut = 1 − trough. |
| Monte Carlo (simulate) | 2,000 futures built from 3-month blocks of the customer's own last 24 months (block bootstrap), the shock applied, starting from today's balance, seeded by account so it's repeatable. Under a shock it reports shares of futures, never “chance”. |
| Calibrated risk (overdraft_risk) | The raw no-shock 12-month break share mapped through a 2-parameter logistic (Platt scaling) fitted 2023 → 2024, judged on 2025. Raw it was 8× too pessimistic because real customers steer back after a bad month. |
| Verdict (verdict) | Can they make 6 months: Yes / No / Only on paper (formula yes, realistic no) / Fragile (yes, but many futures fail); the biggest reason is found by switching each adjustment back and measuring the change in months. |
| Cushion (cushion) | Buffer = alert level = median monthly dip (average − lowest balance); conservative = 75th percentile; top-up now; usual dip day if 70% of months fall within ±5 days. |
| Diagnosis (diagnose) | In order: overdrawn this year → money in last 3 months ÷ prior 9 ≤ 0.75 → new loan payment ≥ 30% of money in → swings (sd of net flow ÷ balance ≥ 0.47) → dip only. Thresholds fixed on 2023. |
| Loan check (loan_check) | Hold for review if, before the loan month, ever overdrawn or in the riskiest 10% by trough. |
| Tiers (book_tiers) | As in Part 4; new accounts scored by the standard deviation of monthly money in, opening month dropped. |
| Survival (survival_cohort, when_overdraft) | Never-overdrawn customers, clock starts in January, event = first day below $0, up to 24 months, censoring when data stops. The per-customer “When?” uses the cohort scored two years before the as-of month, so it never looks ahead. |
| Optimiser (optimize_cuts) | One linear program (HiGHS) over 12 months: the smallest monthly cuts that keep the bad-case (10th percentile) path above the dip; bills and insurance can't be cut; solved once (not MPC). |
| Overdraft cost (overdraft_cost) | $5 per month used + 19% a year (Tangerine's published terms). |

## Statistics: how we know it's true

| Method | Why | Result |
|---|---|---|
| Out-of-time backtest | Features to December, outcome the next year, three separate years | Risk ratio 17.9× / 12.8× / 14.6×; AUC 0.84 / 0.84 / 0.86 |
| Bootstrap confidence intervals | How sure each number is | e.g. realistic beats the formula by +0.058 AUC (95% CI 0.031–0.089) |
| Logistic regression with controls | Is it just balance size? | Odds ratio 16.8 (95% CI 9.5–29.7) after balance, account age, unemployment |
| Benjamini–Hochberg (q = 0.10) | We tried many ideas; correct for luck | 48 of 57 survive; failures shown on screen |
| Platt calibration + Brier score | Make probabilities honest | 1.61% predicted vs 1.61% actual; Brier 0.0142 beats 0.0158 |
| Permutation test (shuffled months) | Do customers steer back? | Yes: +36–37 points vs +18–19 shuffled |

| Method | Why | Result |
|---|---|---|
| Kaplan–Meier + discrete-time hazard | When do overdrafts come? | Flagged: hazard ratio 11.2 and 12.4; 2–4× higher in the first 6 months |
| Champion vs challenger (gradient boosting) | Would ML do better? | 39 vs 34 of 55, not significant (McNemar p = 0.18); its top feature is the trough → keep the trough |
| Fisher exact test | Small-count comparisons | Loan rule p = 0.003; 27 of 27 loans to overdrawn customers defaulted, p = 1.6e-28 |
| Four-fifths rule + Fisher | Fairness by gender and age | Flag-rate ratio 1.12 and 0.86; catch-rate gap flips direction, not significant |
| PSI drift + thresholds | Ongoing monitoring | 10 monitors, 0 breaches |

## The AI, and how it is kept honest

- **Model:** Gemini 3.5 Flash on Vertex AI (the event's Google Cloud project). No key is stored: the machine's or service's Google identity is used. Without it, everything still works offline (rules + templates).
- **Reading twist cards:** Gemini returns typed JSON only; values are clamped; the offline rules read the same card and any disagreement is flagged on the chips before anything runs.
- **Writing:** the brief, the customer message (English/French, no risk percentages, no credit offer) and the plan. Every number must appear in the engine's facts (verify_numbers) and simulation shares can't be called a “chance” (verify_wording); otherwise a template is used.
- **Speed limit:** 6 seconds, then rules or template (one call once hung for 92 seconds; fixed).
- **Measured:** on 16 held-out cards never used for tuning, Gemini 14/16 (88%), offline rules 15/16 (94%). Neither is perfect alone, which is why the chips come first.

## The API

| Endpoint | Returns |
|---|---|
| GET /api/health | OK, which AI is active, 4,500 accounts |
| GET /api/customer/{acct}?asof | Everything on Know the customer |
| POST /api/shock/{acct} | Both runways, why they differ, verdict, simulation |
| POST /api/plan/{acct} | Cushion, costs, optimiser, backup levers, milestones, facts for the AI |
| GET /api/watchlist?asof, /api/book?asof | The call list and the whole-book tiers |
| POST /api/twist | Settings, chips and disagreements from a twist card |
| POST /api/ai/brief, /api/ai/message, /api/write-plan | AI text (checked) or a template |
| POST /api/portfolio, GET /api/evidence | Bank-wide job-loss curve; every tested result |

## Tests and the software lifecycle

| Stage | Check | Count / result |
|---|---|---|
| Engine | pytest -q: acceptance tests per calculation, no-lookahead guards, answer key | 123 fast tests |
| Statistics | pytest -m slow: every evidence claim re-proven | 14 |

| Stage | Check | Count / result |
|---|---|---|
| API ↔ screens | Contract tests; a test parses the React screens for every evidence section they read | 21 API tests |
| AI | Fake-model tests + scripts/eval_ai.py on tuned, dev and held-out cards | 17 + eval report |
| Screens | Streamlit click-through; React type-check and build; axe-core accessibility | 6; 0 WCAG violations |
| CI | GitHub Actions on every pull request: compile, data-free tests, dashboard build | Green on every PR |
| Release | Docker image for linux/amd64 + scripts/smoke.py (8 checks) against it | See Part 8 |
| Operate | Monitoring with thresholds on the Why trust it screen; model card (docs/MODEL_CARD.md) | 0 breaches |

The full list of test cases for each financial calculation is in docs/TEST_PLAN.md.

## Docker and the cloud

One container: a Node stage builds the React app, then a Python stage installs the engine and serves both the API and the built pages with uvicorn on port 8080. The data is copied into the image at build time (never into git). Commands are in docs/DEPLOY.md:

- **Local:** docker build --platform linux/amd64 -t runway:local . then docker run -p 8080:8080 runway:local, then python scripts/smoke.py http://localhost:8080.
- **Cloud Run:** gcloud run deploy runway --source . with 4 GB memory, 2 CPUs, one warm instance, at most three; Vertex permission given to the service's identity.
- **After the event:** delete the service and the project.

# 6. What you built

You (Sukhman) led the engine: the maths, the statistics, the evidence, and the repository. The work in the order it happened:

| Round | What was built | Why it mattered |
|---|---|---|
| Foundation | Judges' formula matching the README exactly; realistic runway (EI cap and end, pensions, dip) | The answer key is right, and our runway beats it out of time |
| The finding | Liquidity trough; 3-year out-of-time backtest; confounders; multiple-testing correction | A discovery, not a dashboard |
| Forecast | Monte Carlo, found 8× too pessimistic, explained (customers steer back), calibrated | Honest probabilities, and a second finding |
| Action | Diagnosis → next best action; cushion and personal alert backtested | Tells the advisor what to do |
| Credit | Loan-approval rule; forward loan payments; Time Machine leak fixed | Business impact in dollars lent; the 93172 story |
| Governance | Monitoring, fairness, model card (OSFI E-23), CI, SQL proof | What a bank needs before production |
| Product | React dashboard, Pitch mode, story strip, call list, whole-book tiers, accessibility | Judges understand it in 5 seconds |
| AI | Gemini on Vertex with typed output, checks, timeout, fallback, measured evaluation | AI that can't invent numbers |
| Depth | Survival analysis, champion vs challenger, new-account score, buffer risk appetite, haircut | Answers “when?”, “why not ML?”, “what about new customers?” |
| Ship | Docker image, Cloud Run guide, smoke test, test plan, this guide | A product, not a notebook |

# 7. Showing it to the judges

## Before you walk up

- Laptop on power; server running; browser at the app, Time Machine on Dec 2024, account 93172.
- Top bar says “AI: Gemini on Vertex AI”. If the Wi-Fi is bad it falls back to offline rules: still read the chips before running.
- Know three accounts by heart: **93172** (the story), **20111** (the answer key: 3.4 / 4.0 / 3.4 months), **20176** (a second twist).

## The 60-second pitch (Pitch mode, press → five times)

1. **The finding:** banks watch the average; overdrafts happen at the low point. Flag 10% → catch 34 of 55 first overdrafts, three years running.
2. **One customer (93172, Dec 2024):** looks fine ($18,503 average, “no shortfall”) → dipped to $238 → the bank lent $167K that month → realistic runway 2.4 months → overdrawn in January. A trough check at approval would have held the loan.
3. **Twist card:** read the verdict, then “what moves the answer”.
4. **Plan:** fix the cause first (call), then the no-risk steps (alert, transfer from savings); a limit comes last, with its real cost.
5. **Trust:** tested on years the model never saw; model health shows no breaches; what failed is listed.

## The live twist (3 minutes, judge picks the account)

1. Type their account in the box → Open.
2. Go to 03 Stress test → paste or type their twist card.
3. Point at the chips: “this is what the AI read; the rules agree (or: here's the disagreement)”. Confirm.
4. Read the verdict sentence out loud, then the biggest reason in months.
5. Scroll to the two runways: “the judges' formula says X, realistically Y, because…”.
6. Go to 04 Make a plan: “first we'd do this; it costs them $0”.

## What to touch, and what not to

| Touch | Avoid unless asked |
|---|---|
| Pitch mode; the story strip; the verdict; the chips; “Do this first”; the whole-book card; the loan card | The optimiser chart (big cuts look unrealistic); the exposure profile (uncalibrated); the full trait table; Streamlit |

## Hard questions, one-line answers

| Question | Answer |
|---|---|
| Isn't the trough just a low balance? | Yes, that's the point: the standard runway and the average both miss it. The ratio and the plain low point are tied; both beat both runway formulas. |
| Why offer credit to risky customers? | We don't lead with it: alert and call first, then a transfer from their own savings ($0); a limit only after a call, at Tangerine's terms. |

| Question | Answer |
|---|---|
| False positives? | The alert pinged 22% of flagged customers who never went overdrawn; 1 in 4 alerts was followed by a real overdraft vs a 1.6% base rate. |
| Is it fair? | Flag rates within the four-fifths band both years; the catch-rate gap flips direction and isn't significant. Gender and age are never inputs. |
| Why not ML? | We tested it: gradient boosting caught 39 vs 34, not significant, and its top feature was the trough. One explainable ratio stays champion. |
| When should the bank act? | In the first 6 months: flagged customers have 11–12× the hazard, highest early. |
| Can the AI make up a number? | No: every number must be in the engine's facts or a template is used. Measured accuracy on unseen cards: 14 of 16. |
| Does it scale? | The core signal is one SQL window query, tested identical to the Python on all 4,500 accounts. |
| Is it causal? | No, it's a signal; the pilot's controlled test is how you'd prove the actions work. |
| Biggest weakness? | Money in is mostly cash, so a job loss assumes all non-pension money stops; and it's 1990s data, which is why the pilot starts in shadow mode. |

# 8. Is it ready to ship?

## Ready for the datathon: yes

| Check | Status |
|---|---|
| All work merged to main; CI green | Yes (PRs #13–#17) |
| Tests: fast, UI, slow, CI mode | All pass |
| Evidence rebuild reproduces evidence.json | Yes (rounding only) |
| Accessibility (WCAG 2 A/AA) | 0 violations on all five screens |
| Phone width | No horizontal scroll at 375 px |
| Offline fallback | Works without Wi-Fi (rules + templates) |
| Docker image and smoke test | See the result in the summary of this round |
| Cloud Run link | Ready to deploy; needs your go-ahead and the organizers' OK on the data |

## Ready for a real bank: not yet (say this before a CTO does)

- No sign-in, roles or audit log; anyone with the link sees every account.
- No customer-data protections (encryption, masking, retention rules).
- One container, data baked in; a real deployment reads from the warehouse (the SQL version is the start).
- Trained on 1990s Czech data: it must be re-validated on Tangerine's own data in shadow mode first.
- No causal proof that the actions reduce overdrafts until the controlled pilot.

# 9. Strict rating and ranking

## Every component: is it useful?

| Component | Score /10 | Verdict |
|---|---|---|
| Liquidity trough early warning | 9.5 | The core finding: simple, tested three years out of time, survives controls |
| Realistic runway (EI cap/end, pensions, dip, new loans) | 9 | Beats the judges' formula out of time; powers the 93172 story |
| Loan-approval rule | 9 | Clear dollar-framed impact for bankers |
| Pitch mode + story strip | 9 | Turns a dense product into a 60-second story |
| Whole-book tiers + call list | 8.5 | Makes it a bank tool, not a single-customer toy |
| Verdict + biggest reason | 8.5 | Best answer to the live twist |
| Cushion, alert, risk-appetite table | 8.5 | Backtested actions with real costs |
| Calibrated Monte Carlo | 8 | Only honest way to say “13.8%”; raw it was wrong, which became a finding |
| Governance: monitoring, fairness, model card, CI | 8.5 | Rare in a datathon; exactly what banks ask |
| AI with number checks + evaluation | 8 | Safe and measured; deliberately small |
| Champion vs challenger | 8 | Answers “why not ML?” with numbers |
| Survival analysis (“When?”) | 7.5 | Rigorous, nice for the professor; doesn't change a decision |
| Diagnosis reasons | 7 | Hold in 2025, not 2024; used to pick actions, not claimed as proven |
| New-account score | 7 | Closes a real blind spot; smaller samples |
| SQL scale proof | 7.5 | Short, convincing answer for data-platform judges |
| Overdraft cost at Tangerine terms | 7.5 | Small, but makes “credit last” concrete |
| Linear-program cuts | 6 | Mathematically right; the cut sizes can look unrealistic (93172 at Dec 2024: about $7,330 a month on average, up to $11,690); folded behind a click |
| Exposure profile | 5 | Uncalibrated, niche; keep it folded away |
| Streamlit backup app | 4 | Insurance only; never demo it |
| Docker + Cloud Run | 7 | Works as one container; untested at load |

## The judges' view

| Criterion (weight) | Score | Why |
|---|---|---|
| Discovery (35%) | 8.5 | Several tested findings: the low point, customers steer back, hidden new loans, the loan rule; failures disclosed |
| Data use (25%) | 9.5 | All 9 files used; leaks caught and tested; out-of-time on three years; fairness; SQL identical to Python |
| Communication (25%) | 8 | Pitch mode and the story strip make it legible; the rest depends on rehearsal |

| Criterion (weight) | Score | Why |
|---|---|---|
| Impact (15%) | 8.5 | Call list, loan rule, real pricing, risk appetite, pilot plan; no “dollars saved” figure (we don't have loss data) |
| Weighted | ≈ 8.7 / 10 | |

## Where we stand

> Honest ranking: on substance this is very likely **top 3 in the Tangerine track, with a real shot at 1st**. I can't see the other entries, so this is a judgement, not a measurement. Few datathon teams ship out-of-time validation, calibration, multiple-testing correction, fairness checks and a model card.

What can still lose it isn't the code: a fumbled live twist, a demo laptop without data or sign-in, a rushed video, or too many numbers in the pitch. The best use of the remaining time is **rehearsal**: Pitch mode five times, and the live twist on 93172, 20111 and 20176 until it's boring.